Progress Report: Exact Sampling of Regular Languages with DLMs

Background and Motivation

I am a third-year undergraduate researcher interested in AI infrastructure and ML systems. I currently work with Binhang Yuan on efficient systems for LLM training and inference, following earlier work with Yi Wu on reinforcement learning systems. My webpage is https://ezoicoder.github.io/.

Jiang, Haghtalab, and Chen [1] study DLM generation through circuit complexity: a DLM with sufficient chain-of-thought space can simulate a depth-d sampling circuit in d rounds, while revision allows workspace to be reused. I focus on a more restrictive question: **exact sampling with no revision and no extra padding/workspace positions**.

Related Work

Looped padded transformers

Svete, Merrill, and Sabharwal [2] write

$$\mathrm{LPT}[p, d, P, T]$$

for looped padded transformers with p precision bits per scalar, embedding width d, P padding positions, and T loop iterations. Let $\tau_1, \dots, \tau_L$ be the layers of a Transformer, with layers $\tau_a, \dots, \tau_b$ forming the loop block. Its computation is

$$H^{(0)} = (\tau_{a-1} \circ \cdots \circ \tau_1)(\mathrm{Embed}(x \circ \texttt{PAD}^P)),$$

$$H^{(t)} = (\tau_b \circ \cdots \circ \tau_a)(H^{(t-1)}), \qquad t = 1, \dots, T,$$

followed by

$$H^{\mathrm{out}} = (\tau_L \circ \cdots \circ \tau_{b+1})(H^{(T)}).$$

Thus, padding supplies parallel workspace, while looping supplies growing sequential depth. The four AC/TC characterizations relevant here are [2]:

$$\text{L-uniform } \mathrm{LPT}[\Theta(1), O(\log N), \mathrm{poly}(N), O(\log^k N)] = \text{L-uniform } AC^k,$$

$$\text{fully uniform } \mathrm{LPT}[O(\log N), \Theta(1), \mathrm{poly}(N), O(\log^k N)] = \text{FO-uniform } TC^k,$$

$$\text{L-uniform } \mathrm{LPT}[O(\log N), O(\log N), \mathrm{poly}(N), \Theta(1)] = \text{L-uniform } TC^0,$$

and, after dropping uniformity and allowing polynomial width,

$$\mathrm{LPT}[\Theta(1), \mathrm{poly}(N), \mathrm{poly}(N), O(\log^k N)] = AC^k.$$

For regular-language recognition, [2] also records

$$\mathrm{Reg} \subseteq \text{L-uniform } \mathrm{LPT}[\Theta(1), O(\log N), 0, O(\log N)]$$

and

$$\mathrm{Reg} \subseteq \text{fully uniform } \mathrm{LPT}[O(\log N), \Theta(1), 0, O(\log N)].$$

These are recognition results: the model only has to return one membership bit, rather than generate an exact joint distribution.

DLMs with and without revision

To keep the comparison unambiguous, I use the following notation throughout:

$$\mathrm{DLM}[p, d, P, T] \quad \text{and} \quad \mathrm{DLM}_R[p, d, P, T].$$

Here p is numerical precision, d is embedding width, P is the number of extra output/workspace positions, and T is the number of parallel decoding rounds. The subscript R means that revision is allowed. For an input of length N, the generated state has $N + P$ positions. The first N positions contain the required output, while the extra P positions may be used as workspace. For unconditional sampling, the visible input is empty. The predictor and position policy are both implemented under the stated (p, d) resource bounds with constant-depth.

Let q denote the predictor and F the position policy. A DLM with revision has the following update process:

```
DLM_R[p,d,P,T]
Initialize z <- M^(N+P)
for t = 1,...,T:
    S <- F(z), where S is a subset of [N+P]
    for each i in S independently:
        z_i ~ q_i(. | z) over {0,1,M}
    all positions outside S stay unchanged
Output the first N positions of z
```

Thus, DLM_R may revise a visible token or turn it back into M (remasking). This is the writable-workspace model used to represent the idealized revision-enabled model of Svete and Sabharwal [3].

Without revision, the policy may select only currently masked positions:

```
DLM[p,d,P,T]
Initialize z <- M^(N+P)
for t = 1,...,T:
    S <- F(z), where S is a subset of {i : z_i = M}
    for each i in S independently:
        z_i ~ q_i(. | z) over {0,1}
    all positions outside S stay unchanged
Output the first N positions of z
```

Once DLM reveals a token, that token is immutable. Both models use conditionally independent parallel predictions within a round; their essential difference is whether earlier decisions can be overwritten.

Let $\mathrm{DLM}_R^{\det}$ denote the
restriction in which every token update is deterministic, i.e., its predictive distribution is one-hot. The recognition results of [3] also impose L-uniformity on the underlying length-indexed Transformer families. The paper builds this condition into its model definition and therefore suppresses it on the DLM side of its notation; I write it explicitly here. Up to logarithmic factors in workspace, its finite-precision, logarithmic-width equivalence is

$$\text{L-uniform } \mathrm{DLM}_R^{\det}[\Theta(1), O(\log N), \mathrm{poly}(N), O(\log^k N)] = \text{L-uniform } AC^k.$$

Their specialized regular-language recognition construction gives

$$\mathrm{Reg} \subseteq \text{L-uniform } \mathrm{DLM}_R^{\det}[\Theta(1), O(\log N), O(N), O(\log N)].$$

The $O(N)$ extra positions form a revisable discrete workspace.

Our Exact-Sampling Problem

For a regular language $L \subseteq \{0,1\}^*$, let
$f_L(x) = \mathbf{1}[x \in L]$ and define

$$\mathcal{G}_{L,N} = \mathrm{Unif}\{(x, f_L(x)) : x \in \{0,1\}^{N-1}\}.$$

The task is to sample exactly from $\mathcal{G}_{L,N}$ using
$\text{L-uniform } \mathrm{DLM}[p, d, 0, T]$: there is no revision and no separate padding or CoT workspace. The partially generated ordered output and its mask pattern are the only persistent state.

One forward round is in AC^0:

$$\mathrm{DLM}[\Theta(1), \mathrm{poly}(N), 0, O(1)] \subseteq \text{randomized-}AC^0.$$

Suppose that, for a regular language $L \notin AC^0$, a no-revision sampler satisfies

$$\mathcal{G}_{L,N} \in \mathrm{DLM}[\Theta(1), \mathrm{poly}(N), 0, T].$$

Exact graph support lets us recognize whether $x \in L$ by testing whether $(x, 1)$ has positive output probability. Composing the randomized-AC^0 updates across T rounds gives a depth-$O(T)$ recognizer for L. The known depth lower bound for suitable regular languages outside AC^0, such as parity, therefore requires

$$T = \Omega\left(\frac{\log N}{\log\log N}\right).$$

The available constructions differ by width:

1. **Logarithmic width.** A binary finite-monoid product tree gives

$$\mathcal{G}_{L,N} \in \text{L-uniform } \mathrm{DLM}[\Theta(1), O(\log N), 0, O(\log N)].$$

2. **Polynomial width.** A $\Theta(\log N)$-ary product tree combines $\Theta(\log N)$ constant-size monoid summaries by a polynomial-size lookup, giving

$$\mathcal{G}_{L,N} \in \text{L-uniform } \mathrm{DLM}\left[\Theta(1), \mathrm{poly}(N), 0, O\left(\frac{\log N}{\log\log N}\right)\right].$$

Hence the polynomial-width bound is tight for regular $L \notin AC^0$, whereas the logarithmic-width case currently lies between $\Omega(\log N / \log\log N)$ and $O(\log N)$. With revision, parity can instead be sampled in constant rounds by $\text{L-uniform } \mathrm{DLM}_R[\Theta(1), \mathrm{poly}(N), 0, O(1)]$, matching the qualitative separation of [1].

Comparison

Here **memory occupation** is the size of the current token/residual representation, excluding model parameters. The **round message** is the state that survives and is visible to the next loop/round. LPTs pass a dense residual workspace, whereas DLMs pass a discrete token sequence.

Task	Model	Revision / state overwrite	memory occupation	state passed to the next round/loop	depth
Regular-language recognition [2]	$\text{L-uniform } \mathrm{LPT}[\Theta(1), O(\log N), 0, O(\log N)]$	Yes	$O(N \log N)$	$O(N \log N)$ unordered	$O(\log N)$
Regular-language recognition [2]	$\text{fully uniform } \mathrm{LPT}[O(\log N), \Theta(1), 0, O(\log N)]$	Yes	$O(N \log N)$	$O(N \log N)$ unordered	$O(\log N)$
Regular-language recognition [3]	$\text{L-uniform } \mathrm{DLM}_R^{\text{det}}[\Theta(1), O(\log N), O(N), O(\log N)]$	Yes	$O(N \log N)$	$O(N)$ ordered	$O(\log N)$
Exact sampling of $\mathcal{G}_{L,N}$	$\text{L-uniform } \mathrm{DLM}[\Theta(1), O(\log N), 0, O(\log N)]$	No	$O(N \log N)$	$O(N)$ ordered	$O(\log N)$
Exact sampling of $\mathcal{G}_{L,N}$	$\text{L-uniform } \mathrm{DLM}[\Theta(1), \mathrm{poly}(N), 0, O(\log N / \log\log N)]$	No	$poly(N)$	$O(N)$ ordered	$\Theta(\log N / \log\log N)$

The central distinction is the state interface: LPTs overwrite dense residual states, DLM_R overwrites discrete workspace tokens, and DLM can only consume masks while preserving every revealed output token.

Outlook: Sampling Regular Languages with Revision

Does sampling mod3 still needs $T = \Omega\left(\frac{\log N}{\log\log N}\right)$. for randomized-$AC^0$.

References

[1] Haozhe Jiang, Nika Haghtalab, and Lijie Chen. *Diffusion Language Models are Provably Optimal Parallel Samplers*. ICLR 2026. https://openreview.net/forum?id=5bkAbueJwM

[2] Anej Svete, Will Merrill, and Ashish Sabharwal. *The Exact Expressive Power of Fixed-Precision Looped Padded Transformers*. 2025. https://anejsvete.github.io/files/fp-lpt.pdf

[3] Anej Svete and Ashish Sabharwal. *On the Reasoning Abilities of Masked Diffusion Language Models*. ICLR 2026. https://arxiv.org/abs/2510.13117